

Synchronizing QC bias ramps with Presto acquisition

PRESTO-8 + AGILENT 33220A | 15 SEPTEMBER 2026

Recommendation

Test a small change to the Presto startup sequence first. Keep the existing AWG gated sawtooth, defer the synchronization in `apply_settings(auto_sync=False)`, and let `get_pixels(..., auto_sync=True)` issue the start event after acquisition has been armed. The installed Presto 2.17.1 source supports this design. It is a credible solution, but its exact hardware timing still needs measurement.

The present code starts the gate before it requests pixels. Host scheduling and instrument command latency lie between those operations. The proposal moves that variable interval before the shared start; it does not try to estimate and subtract it afterward.

Assessment	What can be concluded
High confidence	The current call sequence contains an uncontrolled timing gap. Fixed sleeps or a fixed 50 ms correction cannot make it deterministic.
Strong candidate	A shared internal SYSREF event can plausibly release generation and acquisition together. Public API documentation and installed source support testing this.
Still unverified	Gate behavior before release, the first returned pixel’s timing, startup jitter, and the actual bias waveform at the device.

A synchronization fix also needs correct voltage bookkeeping. The wrapper drops 25 ms, the Presto library drops one initial pixel internally, and folding starts its displayed time at zero. Long records additionally require clock-drift control or measured ramp phase.

Scope and evidence

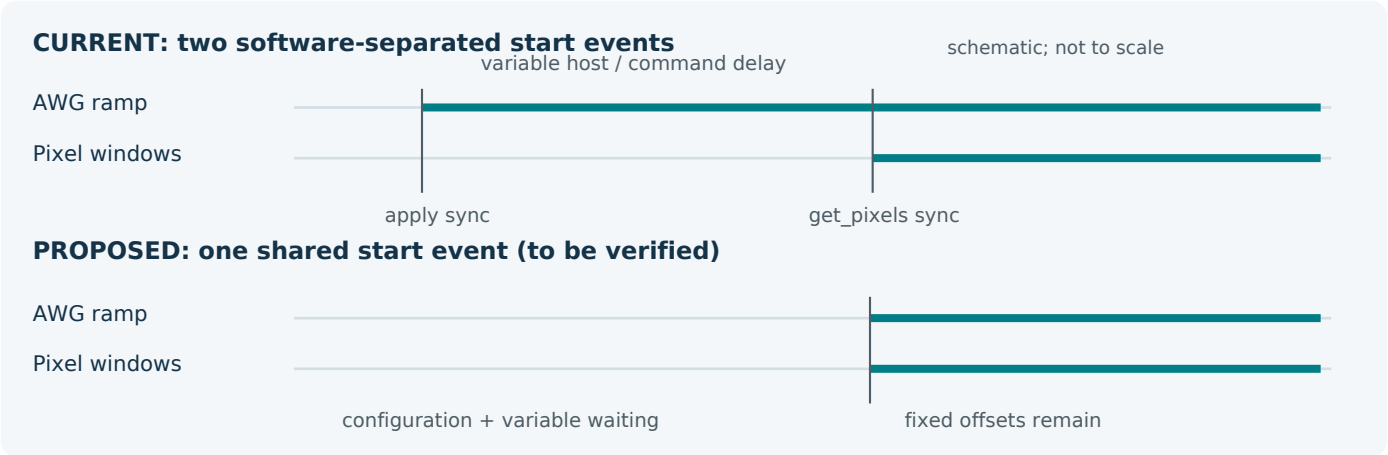
This report audits the repository and official documentation; it does not certify an instrument measurement. No acquisition, instrument connection, firmware update, or measurement-code edit was performed. The local API is 2.17.1; connected instrument firmware and AWG options were not queried. API and hardware evidence are separated from engineering proposals throughout.

Reading order: diagnosis (p.2), minimal fix (p.3), AWG and wiring (p.4), voltage model (p.5), drift (p.6), alternatives (p.7), validation (p.8), implementation boundaries (p.9), sources (p.10).

Where the timing gap enters

The current QC path is **QCTrace.run** → **GateBiasMeasurement._run_gated_ramp** → **TimeStream.run**. StdDevSweep shares this ramp-acquisition machinery, so the same issue applies there. The archived qct_measurement.py uses the same gated-ramp convention.

Stage	Observed code behavior
AWG preparation	_gate_bias.py:278 calls bias.sawtooth(gated=True); the AWG is armed before TimeStream connects and configures Presto.
Trigger setup	timestream.py:366 stages the selected digital outputs with a 30 ms width.
Early start	timestream.py:368 calls apply_settings() with its default synchronization enabled.
Later acquisition	timestream.py:378 calls get_pixels(). An optional on_acquire callback can add more work between the two calls.
Analysis origin	TimeStream saves the untrimmed return data, trims its in-memory arrays, and folding.py:104 creates a new zero-based time axis.



The source shows additional command handling after the early start, including a one-window hardware wait and RF unblanking. Its timing is not a calibrated offset. The observed O(50 ms) variability is consistent with this architecture; this audit cannot identify how much comes from each host, network, or server operation.

The wrapper’s “external_trigger” is a **Presto output** controlling another instrument. It does not mean the Lockin receiver is waiting for an external TTL acquisition trigger. The documented get_pixels return contains quadratures and frequencies, without a per-pixel trigger timestamp. [1]

The smallest credible fix

Proposed experiment — not applied to the repository

```
# Finish tuning and input/output-group configuration first.
# Establish a low gate; prepare and arm the AWG deliberately.
lck.set_trigger_out(trigger_states, width=0.03)
lck.apply_settings(auto_sync=False)
# No intervening operation may issue a synchronization event.
pixel_dict = lck.get_pixels(pixel_counts, auto_sync=True)
```

SYSREF is Presto's internal synchronization event. In the proposal, variable Python or network delay before `get_pixels` changes when the experiment begins, while the hardware event establishes the relative start of the ramp and acquisition.

Why this is supported

The public API describes `apply_settings(auto_sync=False)` as deferring generation synchronization; `get_pixels(auto_sync=True)` internally synchronizes acquisition. It does not explicitly specify the combined sequence's first-gate-edge to first-returned-pixel delay. [1]

The installed source is more informative: `lockin.py:283–307` stages trigger settings and conditionally emits `hardware.sync()`; `:877–882` arms pixel acquisition and then emits `hardware.sync()`. Both paths refer to SYSREF. Deferred RF configuration also receives `do_sync=False` (`:222–238`). This supports a common release, but Python source is not a substitute for FPGA timing verification.

Conditions for a valid test

Start from a verified low gate. A fresh Python object does not establish the electrical idle state. Complete configuration while the AWG cannot respond to unwanted edges, then arm it with the line low. Check the full preparation sequence for unintended syncs.

Retain real pending IF settings. In 2.17.1, `set_trigger_out` does not itself mark IF configuration dirty. Initial `TimeStream` setup does; a later trigger-only update may need a public setter such as `set_df(lck.get_df())` before `apply_settings`. Do not rely on a silently unsent update.

Measure startup settling again. RF generation may now start alongside acquisition. The old leading-data discard may still be useful, but its required duration and phase effect must be measured. Keep the normal immediate cleanup behavior; do not change every `apply_settings` call to defer synchronization.

Decision: accept this method only if a recorded timing witness stays at the same pixel position when deliberate host delays are inserted before `get_pixels`. See p.8.

AWG behavior and the physical link

What the manufacturer documents

The 33220A's gated burst runs while the gate is active; after deassertion it finishes the cycle and holds its start-phase voltage. Trigger-source selection is ignored in this mode. Its front Sync output follows the gate/burst envelope, not every ramp cycle. Continuous and infinite-count burst operation provide cycle timing there. [3] (pp.68-69, 107-108).

Ramp phase zero is the rising offset crossing; phase units are configurable. Trigger specifications list latency below 500 ns and 6 ns RMS jitter, with TTL pulses longer than 100 ns into a DC-coupled input above 10 k Ω . These are instrument specifications, not end-to-end alignment guarantees. [3] (pp.112, 348).

What that means for this measurement

Keep the existing gated mode for the first test. Switching to an edge-triggered burst while retaining the early Presto start would retain the software timing gap. A bus trigger, including *TRG, issued just before get_pixels would also leave two software-controlled starts.

The wrapper chooses 100% ramp symmetry, phase 180°, and a default offset of half the peak-to-peak amplitude. For an ideal rising sawtooth, 180° lands at rollover. Calibrate the actual first ramp and held level instead of assigning an endpoint from that discontinuity. A future implementation should explicitly select degree units before programming phase and read back the state it depends on.

Wiring to verify at the receiving connector

```
Presto selected DIGITAL OUT ---> 33220A rear TRIG / BURST GATE
33220A analog OUTPUT      ---> existing device bias line
Scope: observe gate and actual analog ramp together
```

Presto's datasheet lists a 3.3 V digital output with 50 Ω output impedance. Establish the loaded voltage, overshoot, cable termination, and accepted AWG input levels at the actual connector; do not infer compatibility from "TTL" alone or assume whether 3.3 V denotes open-circuit or terminated operation. [2] (p.4).

The 30 ms gate width is the existing configuration, not a verified continuous-level command. At 50 kHz, windows are 20 μ s apart. The repository infers that overlapping triggers keep the line high and explicitly says this has not been scope-verified. Look for narrow low glitches and verify rearming and gate-off behavior. At much lower df, the same width can produce separated gates.

Assigning voltage to each pixel

Repeatability and absolute voltage are separate requirements. A fixed delay can be calibrated. A variable delay must be eliminated or measured per record. Even perfect timing cannot turn an averaged pixel into an instantaneous measurement.

Use a calibrated time origin

Let f_s be the tuned Lockin pixel rate, f_r the measured ramp frequency, and $D = \text{round}(\text{discard_start_ms} \times 10^{-3} \times f_s)$. Let t_c be the measured center time of returned raw pixel 0 relative to the gate edge. For retained pixel j :

$$t_{\text{center}}(j) = t_c + (D + j) / f_s$$

The installed library requests and removes one initial pixel when no FIR is used (lockin.py:794–820, :905). This does **not** prove $t_c = 1.5/f_s$: the discarded pixel may represent pipeline or partial-window behavior. Measure t_c or obtain its definition from Intermod. Folded data must preserve D and t_c rather than treating retained sample zero as the gate edge.

At the lab defaults, $D = 1,250$ and $D/f_s = 25$ ms. A 500 Hz ramp has a 2 ms period, so trimming alone moves the origin by **12.5 periods: half a cycle modulo the ramp**. Hardware synchronization does not remove that phase shift.

Ideal ramp model, away from reset

$$\begin{aligned} u(j) &= \text{frac}[f_r * (t_{\text{center}}(j) - \tau) + u_{\text{start}}] \\ V_{\text{source}}(j) &= V_{\text{offset}} + V_{\text{pp}} * (u(j) - 0.5) \end{aligned}$$

Here frac keeps the fractional part, τ is the calibrated gate-to-ramp delay, and u_{start} is the measured initial phase. This formula assumes a 100% rising sawtooth. Other symmetry settings need a piecewise rise/fall model. Device voltage additionally depends on bias-line gain, offset, filtering, and propagation delay; a fixed delay is inadequate if the line distorts the ramp.

Each unsummed pixel integrates over roughly $1/f_s$. On a locally linear ramp the voltage traversed in that interval is $V_{\text{pp}} f_r / f_s$. For the illustrative 2 Vpp, 500 Hz ramp at 50 kHz, this is **20 mV per pixel**. A midpoint voltage is a useful label, not an exact description of all voltages contributing to the result. Exclude reset-spanning and unsettled windows or model their response.

Away from reset, a timing uncertainty δt produces approximately $\delta V = V_{\text{pp}} f_r \delta t$. Thus a 1 mV timing-error budget in this example requires about 1 μs timing knowledge. This is distinct from the 20 μs integration width.

Prevent phase drift during the record

One start event does not lock two clocks

A relative clock error ϵ accumulates a time slip of approximately ϵT over a record of duration T . As an illustrative calculation, 20 ppm gives 8 μs over 0.4 s and 200 μs over 10 s. These are examples, not measured errors of this lab's AWG. The 33220A's specified one-year internal reference accuracy is ± 20 ppm. [8]

If Option 001 is installed, share a 10 MHz reference. Its input accepts 100 mVpp–5 Vpp and requires lock acquisition before the run. [8] Presto can supply the reference; API 2.17 changed that output to disabled by default, so explicitly configure `hardware.set_output_reference(10e6)`. [4] [6]

Use one reference master. With Presto as master, keep its internal reference selection and feed its reference output to the AWG reference input. If another master already supplies the rig, distribute that clock consistently. Verify the installed option, signal level, and lock status. Reference locking controls rate drift; the proposed start event still establishes sweep phase.

Rounded folding periods cause a different drift

Current folding uses $N = \text{round}(f_s/f_r)$ samples per period. The timing error at the beginning of block m is:

$$\text{Delta}_t(m) = m * [N / f_s - 1 / f_r]$$

For $f_s = 50$ kHz and $f_r = 300$ Hz, $N = 167$. The block boundary slips by 6.667 μs per period; at block 199 it has slipped by 1.327 ms, about 40% of a ramp. A shared reference does not correct this arithmetic mismatch. Smearing depends on feature width; there is no universal contrast-loss percentage.

Choose one defensible analysis method

Commensurate acquisition: after tuning Presto, choose an achievable AWG rate close to f_s/N , verify the physical ratio, and check residual drift over the full record. Construct capture length using the tuned rate and the intended complete periods.

Measured-phase folding: derive ramp phase from a witness or calibrated physical frequency, preserve the initial phase and discarded samples, and interpolate or bin against phase. Reject incomplete cycles and reset transients. A SCPI frequency query verifies the programmed setting; it does not independently measure the oscillator's true frequency.

Alternatives if the small change fails

Method	Why consider it / what remains
Record actual ramp phase	Retains continuous Lockin. Recovers voltage labels despite asynchronous starts and measures drift. Requires an independent timing witness and calibrated acquisition path.
Pulsed sequence	Schedules a marker and acquisition in the same hardware sequence. Requires a measurement redesign and a budget for bandwidth, memory, and averaging.
External SYSREF	Vendor-supported synchronization hardware could release both armed operations. Requires confirmed hardware, routing, and arm/start semantics.
Presto DC waveform	Removes the separate AWG from the bias path. Its waveform-start relation to Lockin pixels is not established by the reviewed public documentation.

Most robust diagnostic: observe the ramp in the pixel record

An engineered witness can compare the real bias ramp against a known threshold, use that comparator to switch an RF carrier, and record the carrier envelope on a spare Presto input. A unipolar sawtooth gives a recurring reference crossing. Calibrate comparator and RF-switch delay, choose a resolvable envelope duration, and choose a threshold crossing late enough to remain visible after the initial-pixel discard. Capture gate prehistory on the scope. This is a proposed measurement circuit, not existing wrapper functionality.

The documented Presto RF inputs are AC-coupled RF paths; directly digitizing a 500 Hz DC ramp or a TTL line through them is not a valid general solution. An RF witness, or a suitable DC digitizer synchronized to a recorded acquisition reference, is needed. [2] (p.3).

Hardware-timed acquisition

Pulsed exposes output_digital_marker and store on the sequence time grid; long flat matching and downsampling also merit evaluation. [5] The older hardware sheet gives a 524 μs full-rate contiguous store window. [2] Treat that as a baseline: current downsampling changes bandwidth and duration. Check usable IF bandwidth, channels, raw storage, integration, and repetition limits for the actual QC experiment.

An illustrative redesign would arm a finite AWG burst, issue one sequenced marker, and acquire at a calibrated offset. Triggered bursts accept 1–50,000 cycles or infinity; triggers during an active burst are ignored. [3] (pp.110, 113). Ensure rearm margin and preserve individual cycles if stochastic parity dynamics matter; averaging them in hardware changes the measurement.

Bench validation that can falsify the proposal

No bench test was run for this report. The following protocol is intended to distinguish a real common start from a merely repeatable-looking software sequence. Begin on a suitable test load and preserve raw calibration data.

1. Characterize the physical signals

Observe the gate at the AWG connector and the analog ramp simultaneously. Record idle level, first-ramp phase, trigger-to-ramp delay, low glitches, actual period, and behavior when the gate falls. Test the present 30 ms overlap configuration at the intended df values. Establish adequate settling and rearm intervals.

2. Put timing evidence inside the acquisition

Use a calibrated witness visible in the same returned Lockin pixels as the readout (p.7). A scope showing gate and AWG output alone cannot locate the first returned pixel. A spare scope channel or another marker emitted at every demodulation window also does not automatically identify which window get_pixels returns.

3. Insert deliberate host delay

Compare the current sequence with the proposal. After apply_settings(auto_sync=False), insert randomized waits spanning 0–100 ms before get_pixels, including values that are not integer ramp periods. Verify that the gate and ramp remain inactive while waiting and that the first recorded ramp event keeps the same sample position after release. A changing position falsifies the intended common-start behavior.

4. Measure repeatability and drift

As a first validation set, take at least 100 starts across reconnects and representative host load; then exercise the longest planned record and several rates. Capture pre-start baseline on the scope and enough post-start time to expose missed cycles. The simultaneous Lockin start does not supply pretrigger pixels. Report mean offset, standard deviation, range, outliers, cycle slips, and phase slope within records. Use absolute first-event timing as well as phase modulo one period; a whole-cycle error can otherwise be invisible.

5. Set acceptance from the scientific requirement

$\text{allowed timing uncertainty} = \text{allowed voltage error} / (V_{pp} * f_r)$

The witness's own uncertainty must be below that allowance. One-pixel repeatability at 50 kHz establishes only a coarse 20 μs bound, not 1 μs accuracy. Fit sub-window timing only with a calibrated response model and uncertainty estimate, or validate at a suitable higher rate. Separately account for finite-window averaging and bias-line distortion.

Pass: no early ramp, stable calibrated pixel-to-ramp offset, no unexplained cycle slips, and acceptable full-record drift. **Fail:** retain the raw evidence; use measured-phase acquisition or a validated hardware-sequenced redesign instead of adding a fitted constant sleep.

Boundaries of a later implementation

Changes to consider only after the timing test

Scope a future synchronization option to QC ramp startup in TimeStream and its callers. Preserve immediate output muting and AWG output-off behavior on success and failure. Include preparation and deferred arming in cleanup coverage. Account for changed on_acquire semantics: it currently promises a callback after generation has begun, which would no longer be true in the deferred-start path.

Save the synchronization method, actual API and connected firmware versions, tuned df, requested/raw/retained pixel counts, D, measured t_c , calibration uncertainty, AWG settings and reference state, and phase/drift model. Keep untrimmed data. Save a voltage or phase coordinate with the folded trace so later analysis does not silently reinterpret its zero.

Two tempting routes that remain unproven

AWG Sync → Presto digital input: ordinary digital-input triggering is documented for Pulsed. [4] It is not interchangeable with Lockin's external SYSREF mechanism. Merely setting `get_pixels(auto_sync=False)` does not select a TTL input. A vendor-confirmed external synchronization design needs explicit clock, SYSREF, and readiness handling.

Built-in DC waveform: Presto 2.17 added repeating DC-bias waveforms, including a sawtooth. [6] [7] The reviewed interface does not establish the start phase relative to returned pixels. Before replacing the AWG, verify that timing, update cadence, reset transients, and analog bandwidth meet the requirement. Its DC output impedance is 1 k Ω , so loading also differs from the AWG path. [2]

Questions for Intermod, if bench evidence is ambiguous

Does the proposed `apply_settings(auto_sync=False) → get_pixels(auto_sync=True)` sequence release generation and pixel integration from the same SYSREF? Is digital output guaranteed low before that event? What is the first returned pixel's exact integration interval, including the skipped invalid pixel? Is trigger width greater than a window a supported continuous gate? How should trigger-only updates be applied in 2.17.1? Can a DC waveform reset its phase on that same event?

These are suggested technical questions; no message was sent to the manufacturer.

Conclusion

There is a technically grounded path forward. The deferred-generation change is the best first test and could remove the O(50 ms) variable start gap with limited wrapper changes. Reliable voltage assignment additionally requires a measured pixel origin, preserved trim phase, and control or measurement of drift. I cannot honestly claim that synchronization is fixed until the bench tests establish those properties.

Sources and audit trail

Official documentation accessed 15 September 2026. Numbered citations are clickable. Manufacturer facts support the design; proposed circuits, error calculations, and test criteria are engineering analysis. Page numbers for the 33220A guide are its printed numbers; the PDF viewer page number is one larger.

[1] Presto Lockin API, 2.17.1

<https://www.intermod.pro/manuals/presto/source/lockin.html>

apply_settings, get_pixels, set_trigger_out; separate auto_sync controls.

[2] Presto hardware datasheet, 2024-07-15

https://intermod.pro/res/docs/presto_spec_sheet.pdf

pp.2-5: pulse resources, analog inputs, digital ports, clocks and DC outputs.

[3] Agilent 33220A User's Guide, edition 4, May 2007

<https://www.keysight.com/us/en/assets/9018-04437/user-manuals/9018-04437.pdf>

pp.68-69, 107-114, 144-145, 223-230, 347-348: Sync, burst, phase, reference and trigger behavior.

[4] Presto Hardware API, 2.17.1

<https://www.intermod.pro/manuals/presto/source/hardware.html>

set_output_reference, TriggerSource, setup_dcbias_waveform.

[5] Presto Pulsed API, 2.17.1

<https://www.intermod.pro/manuals/presto/source/pulsed.html>

output_digital_marker, store, downsampling, flat matching and run.

[6] Presto changelog

<https://www.intermod.pro/manuals/presto/CHANGELOG.html>

2.12.0 auto_sync; 2.17.0 DC waveforms and reference-output defaults.

[7] Presto DC-bias waveforms, 2.17.1

<https://www.intermod.pro/manuals/presto/source/dcbias.html>

DcBiasWaveform interface and Lockin example; no explicit pixel-origin guarantee.

[8] 33220A product datasheet, 5988-8544

<https://www.keysight.com/us/en/assets/7018-01144/data-sheets/5988-8544.pdf>

Reference accuracy and Option 001 electrical specifications.

Local code inspected

Repository: C:/Users/AQD/Documents/GitHub/daq

Commit: 8f1e4d7700fb5e12fff5c3a9d299d88ab226465e (clean before report creation).

daq/measurements/timestream.py:106-124, 269-293, 296-419

daq/measurements/_gate_bias.py:162-200, 258-288

daq/measurements/qc_trace.py:240-272, 277-380

daq/measurements/sweep_std_dev.py:625-691

daq/instruments/function_generator.py:180-247; daq/analysis/folding.py:39-106

Installed source root: C:/Users/AQD/.conda/envs/presto/Lib/site-packages/presto

version.py:62-75; lockin.py:222-314, 548-598, 794-905; hardware.py:665-731.

The package identifies API 2.17.1 and expected conductor 0.11.5 / FPGA 48. These are not a query of the connected Presto. Repository CLAUDE.md was consulted for environment and hardware context; manufacturer documentation takes precedence over its older assumptions.

LED timing: conclusion and scope

APPENDED 15 SEPTEMBER 2026 | ORIGINAL PAGES 1-10 PRESERVED

The software-start explanation is well supported. Current LED pulse measurements enable the DC2200 through USB inside on_acquire, then request Presto pixels. That callback contains multiple transactions and leaves a variable delay between two independently started instruments. It improves proximity to acquisition; it cannot establish a hardware timing relationship.

Recommended solution for continuous multitone readout

Configure DC2200 TTL mode and generate the entire pulse train in hardware. A promising concrete option is the NI PCIe-6361 already used in historical lab notebooks, if the card and a counter are available. Pre-arm its counter; start it from an acquisition-aligned Presto digital edge; derive its timing from the Presto reference clock where electrical interfacing permits. This preserves the continuous Lockin readout. Pages 14–15 specify the proposal.

The deferred Presto start proposed on p.3 remains a prerequisite to test. That change alone does **not** synchronize a DC2200 internal pulse train that is still enabled by software. For illumination over the entire acquisition, pre-armed TTL gating is a simpler case; short repeated flashes require an actual pulse waveform.

Recent saved configuration	Value
Presto / bias ramp	100 kHz pixels / 5 kHz Agilent ramp
LED waveform	100 μs ON + 10 ms OFF; period 10.1 ms (≈99.01 Hz)
LED drive / record	0.499 A, terminal 2; continuous train; 1 s requested record
Trimming	0.1 ms wrapper discard; a further 25 ms for plotting

These values come from executable notebook settings and one saved manifest, not its stale introductory prose. The older 10 μs / 2 Hz recipe is a separate supported use case, not the assumption for this appendix.

Evidence limits: source and saved metadata were inspected; no notebook or instrument was run. NI hardware availability, current routing, DC2200 firmware, and optical latency remain unverified. This appendix proposes a testable implementation rather than claiming the synchronization is already fixed.

What actually happens between the calls

```

TimeStream.run:
  apply_settings()          # currently starts the Presto gate
  on_acquire():
    SYST:ERR?              # drain old LED errors
    OUTPUT1:STATE ON       # request LED output enable
    optional transcript flush
    SYST:ERR?              # check the enable command
    callback metadata work
  get_pixels(...)          # Presto acquisition request + sync

```

This path is visible in `timestream.py:368–378`, `dc2200.py:147`, and `_visa.py:608–650`. Error draining normally performs one query and can perform more. At least one error-query round trip follows the enable command. USB delivery, DC2200 command processing, host scheduling, and subsequent Presto commands all contribute. Their exact contribution to physical timing cannot be inferred from statement order.

A measured host duration, with a narrow interpretation

Saved run 20260915T204734Z_3eb90b91 records host timestamps immediately before and after the LED output setter: 1789505260.7048965 and 1789505260.7246845. Their difference is **19.788 ms**—longer than one 10.1 ms LED period. This is one host-call duration, not the LED-to-pixel delay, a jitter distribution, or proof that a particular number of optical pulses was missed.

Existing logging and analysis can conceal the problem

KickNLED timing field: the August 3 and September 2 wrappers take both timestamps used for `visa_write_s` after `led.output=True` returns. That field excludes the enable call and cannot measure its duration or synchronization jitter. More recent wrappers bracket the call correctly, but still measure host activity.

Estimated overlays: the multitone helper assumes a first pulse at raw sample zero plus a user offset; `LED_PULSE_OFFSET_S` defaults to zero. Its `estimated_LED_pulse_times.csv` is a prediction from that assumption, not captured pulse timing. Each independently restarted segment needs its own measured phase.

Post-record exposure: continuous internal pulsing can continue while `TimeStream` transfers, processes, and saves data, because the caller disables the LED after run returns. Finite hardware scheduling should define the intended final pulse as well as the first one.

`daq/time_utils.py` formats local date/time for filenames. It does not timestamp hardware samples. Similarly, a host `t0_unix` and a settings snapshot are not observations of LED edges.

DC2200 modes use different timing paths

Mode	Timing source and rear MOD IN/OUT role
PULS / PWM	Internal pulse timing; rear SMA provides a TTL monitor output.
TTL	External level controls light: low is off, high applies the selected current.
CC	Constant current when enabled; no externally defined pulse sequence.

The reviewed Thorlabs manual documents these distinctions and no external arm/start command for the internal pulse train. INITiate in its SCPI section initiates internal measurements; it is not a documented pulse-train trigger. [9] (pp.16–20, 45–48).

Therefore connecting a Presto output to the MOD connector while retaining PULS mode is not a supported synchronization method—and can connect two outputs together. For external timing, configure TTL and supply every intended ON and OFF transition. Read back mode before deciding cable direction.

The TTL arming order matters

One current notebook TTL branch calls apply_settings before the callback enables the LED. If the modulation line is already high, USB output-enable timing still determines light onset. Instead, establish a verified low input, enable the TTL output stage while it remains low, then allow a later hardware edge to command the first pulse. Software can take a variable time to arm an idle instrument without shifting that later edge.

Electrical timing is not optical timing

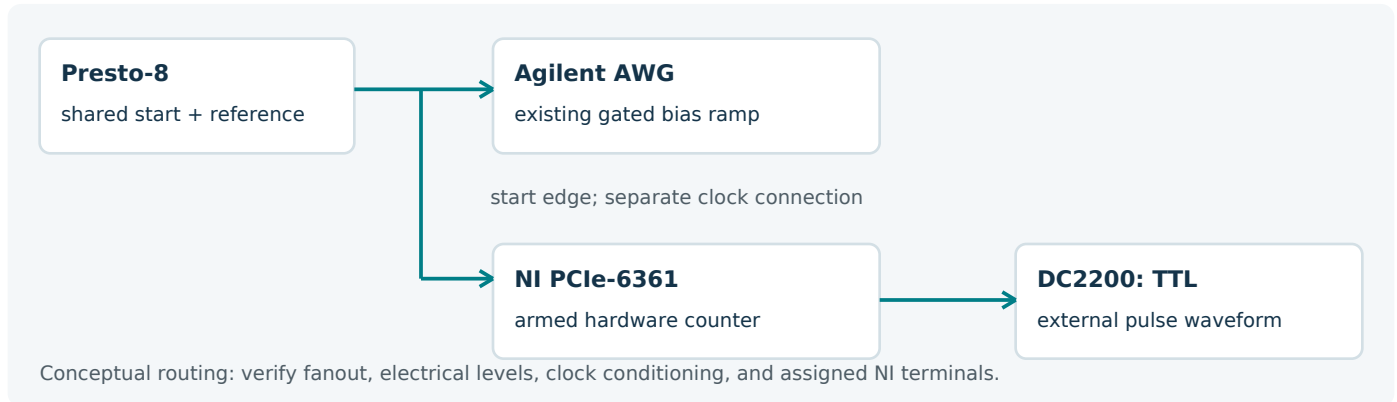
Specified TTL input levels are 0–0.8 V low and 2–5 V high, with 10 kΩ input impedance. The manual’s TTL bandwidth characterization depends on drive current and resistive load; its 250 kHz analog small-signal bandwidth describes another mode. It supplies no numerical USB-start latency or optical-jitter guarantee. [9] (pp.19–20, 67).

Measure light with a photodiode at the relevant current, LED head, cable, and temperature. A command monitor says when modulation was requested; it does not establish optical pulse amplitude, width, or delay. In particular, do not infer the 0.499 A optical waveform from a low-current or resistive-load bandwidth test.

Check the actual loaded Presto/NI voltages and overshoot at receiving connectors. Use appropriate fanout and clock conditioning; neither “3.3 V” nor “TTL” by itself specifies the complete interconnect. Changing to TTL also changes how current is programmed, so verify the optical pulse energy as well as timing.

Documentation scope: the retrieved manufacturer manual is v1.7 (11-Apr-2022), corroborated by v1.6. The driver cites v1.8, which was not located during this review. Absence of an external-start feature is a finding about the reviewed documentation, not a claim about every possible firmware revision.

Concrete proposal: an NI hardware counter



Historical NIDAQ notebooks name an NI PCIe-6361 and use /Dev1/PFI0 as a digital start input. Its counter hardware supports a finite pulse train with a hardware start and programmable initial delay. [10] [11] The installed Python environment also has nidaqmx; neither observation proves a card or counter is currently available.

Preparation and release order

1. Disable LED output and establish a low Presto start line. Configure readout and AWG bias; prepare the NI counter with idle LOW, 100 μ s high, 10 ms low, a deliberate first-pulse delay, and a finite pulse count. Choose the delay from measured acquisition settling and the count so the last pulse ends inside the desired record.
2. Configure the NI rising-edge start trigger and clock source. Start its task so it waits for the edge. Configure DC2200 TTL and enable its output while the counter output is still low. Complete queries and other preparation before release.
3. Stage the Presto start/gate output and call `apply_settings(auto_sync=False)`, with no unintended intervening sync. Let `get_pixels(..., auto_sync=True)` issue the shared event. As on p.3, verify that this actually creates the first digital edge at a fixed offset from returned pixels.
4. The NI counter generates the flashes after that edge; software timing no longer starts each train. A spare Presto digital output can carry the same start signal as the AWG output, or use verified fanout. DC2200 receives the NI pulse waveform, not the steady AWG gate.

Counter settings to implement later

```

C0 channel: Dev1/ctr0          # illustrative device/routing
High / low time: 100e-6 / 10e-3 seconds
Initial delay: measured settling margin
Implicit timing: FINITE, selected pulse count N >= 2
Start trigger: rising edge on /Dev1/PFI0
Retriggerable: False; output idle LOW
Task.start(): arm before the Presto start edge
  
```

NI's same counter Gate resource cannot simultaneously act as start and pause input. A falling start signal therefore must not be assumed to stop an already started train. Use a finite schedule plus explicit LED-off/counter cleanup on every exit; add separate hardware inhibit if immediate abortion is required. [11] (pp.7-28-7-31).

Clocking, alternatives, and common traps

Remove drift as well as the initial software gap

NI specifies 50 ppm timebase accuracy, external counter clocks up to 25 MHz, and 10/20 MHz reference locking through PCIe PFI terminals. [10] At an illustrative 50 ppm relative error, start-only synchronization can slip 50 μ s in 1 s or 0.5 ms in 10 s. These are error-budget calculations, not measurements of this card.

One implementation route is a verified, electrically conditioned 10 MHz Presto reference into the NI counter’s external timebase. For the proposed waveform this gives 1,000 high ticks and 100,000 low ticks. Set the timebase source and its actual rate explicitly; a rate property alone does not connect a clock. Alternatively, use the supported NI reference-lock route. Presto’s reference output must be enabled explicitly in API 2.17. [4] [6]

This locks NI timing to Presto, not automatically to the Agilent. Keep the AWG reference and phase requirements from pp.4–6. Hardware tick resolution is also not a claim about optical-edge accuracy.

Alternative	Assessment
Direct Presto gate → TTL LED	Suitable for an illumination window after common-start validation. Existing 30 ms width is not the requested short repeated flashes.
Lockin marker / sum settings	Markers occur per raw or sum window, with shared width/delay. Summing alters the returned data rate; it is not a free LED timer.
Hardware divider / sequencer	Can count a Presto-derived clock and shape flashes. Requires deterministic reset, arming, and separate treatment of the AWG gate. NI is one concrete realization.
Presto Pulsed mode	Hardware-scheduled markers and acquisition are available, but require redesign of continuous multitone capture, bandwidth, and storage.
Record actual LED edges	Retains internal PULS and permits measured alignment afterward; does not make physical starts simultaneous. See p.16.

At 100 kHz, a 100 μ s marker exceeds the 10 μ s raw window and cannot create 100 μ s ON / 10 ms OFF. The shared delay/width also prevents independently configuring a long AWG gate and short LED clock. At 50 kHz, $nsum \leq 1023$ cannot directly yield the older 2 Hz recipe. These conclusions follow from the Lockin interface and installed validation. [1]

Removing error checks, adding sleeps, using threads, or subtracting a fixed host duration can change average latency; none establishes a shared hardware start. Keep error handling outside timing-critical release where possible rather than using its removal as synchronization.

Measure the light and preserve its time axis

Separate three quantities

Command time: a NI output edge or DC2200 MOD monitor edge. **Optical time:** the photodiode sees emitted light. **Device-response time:** the measured resonator/parity response, including its physical and filtering delays. A detector response must not be treated as an independent timing reference when that response is the scientific quantity being studied.

For characterization, record the Presto start reference, LED command, and photodiode in one scope capture. To map those edges into pixels, additionally establish the first returned pixel's position through the calibrated in-record witness described on p.8, or another validated mapping to the acquisition hardware. A scope trace of two external signals alone does not identify pixel zero.

An appropriately synchronized NI acquisition can record the photodiode/monitor; counter timestamps or digital capture must also reference the Presto acquisition event. Account for separate-device clock drift and input-channel timing. Ordinary Presto RF inputs are not DC/TTL digitizers, and no ordinary Lockin digital-input timestamp stream was established in the reviewed API. [2] [1]

A useful event model for the proposed hardware sequence

```
t_light(k) = t_gate + delay_initial + k*T + delay_optical + error(k)
t_pixel(j) = t_gate + t_center_raw0 + (D + j)/f_s
t_on_plot(k) = t_light(k) - t_gate - t_center_raw0 - D/f_s - trim_plot
```

Use the tuned f_s . D is the exact wrapper discard in samples. t_center_raw0 is the calibrated raw pixel-center offset, including the library's initial-pixel handling. For internal PULS, replace the scheduled edge times with measured edges or a fitted per-segment epoch and period. Preserve absolute pulse count where possible; phase modulo a period cannot reveal missed first pulses.

Sampling and trimming are material here

At 100 kHz, a pixel integrates over 10 μ s and a 100 μ s pulse spans about ten windows, depending on phase. The older 30–60 μ s pulses at 10 kHz are sub-window events. Even `discard_start_ms=0` does not restore the initial pixel removed internally by Presto.

For the recent saved run, 0.1 ms wrapper discard plus 25 ms plotting trim shifts the displayed origin by about 25.1 ms. The notebook helper already tracks those trims; its remaining unmeasured first-pulse assumption is a different issue. An attach snapshot with `led_output=False` can legitimately precede an illuminated record, because the callback enables the LED afterward.

Validation and bias-phase interpretation

A hardware test with a clear failure condition

Baseline: retain the present PULS method and capture command monitor, optical output, and acquisition reference over repeated starts. Host before/after timestamps are useful diagnostics, but label them as host durations. Include different segment lengths and USB/host load; do not derive jitter from one call.

Candidate: pre-arm the counter and TTL LED. Insert randomized 0–100 ms waits before get_pixels. There must be no light while waiting. Measure first optical edge, width, pulse count, and the optical-to-pixel offset. A change of offset with the inserted wait falsifies the intended common-start design. Repeat after reconnects, errors, and rearming, with the line low before every new start.

Full-record check: test at least 100 starts and the longest intended record; measure drift across all pulses, not only pulse one. Verify the last pulse and darkness during unrecorded saving/idle periods. Test the actual terminal/current and compare optical energy between internal PULS and external TTL. Set the allowable offset/jitter from the experiment; for a 10 μ s requirement, the witness uncertainty must be substantially smaller than 10 μ s.

Perfect synchronization does not imply identical bias phase

Current nominal settings:

LED period = 100 μ s + 10 ms = 10.1 ms
 Bias period = 1 / 5 kHz = 200 μ s
 LED period / bias period = 50.5
 LED ON time / bias period = 0.5

Thus successive flashes alternate by half a ramp cycle, and each illumination spans half a bias sweep. This is a property of the requested waveform, not evidence that synchronization failed. Exact ratios require actual clock rates and shared-reference control.

If every flash should begin at the same ramp phase, an integer number of bias periods is needed: for example, a 10 ms LED period with 100 μ s ON would require **9.9 ms OFF**. That would change the repetition rate to 100 Hz and is an experimental choice. It would still illuminate across half a ramp; a shorter pulse or a bias hold would be needed to approximate one bias voltage.

What a later implementation should record

Save mode, terminal/current, trigger routing, counter source/rate, pulse counts and initial delay, actual df, raw/retained sample origins, both trims, calibration uncertainty, and measured electrical/optical edge times. For segmented runs retain each segment's hardware epoch and real gaps. Concatenating only recorded durations must not imply the source ran continuously on that time axis.

LED appendix: evidence and next step

New primary-source references

[9] Thorlabs DC2200 Operation Manual, v1.7, 11-Apr-2022; manufacturer PDF hosted by distributor

<https://onsnab.ru/catalog/pdf/xnsng8stwznu13thk1i6vh6xoz12ulq.pdf>

Manufacturer-authored PDF from a distributor mirror. Verified v1.7, 11-Apr-2022; printed pp.16–20, 36, 45–48, 66–67. PDF page number = printed page + 2.

[10] NI PCIe-6361 / PXIe-6361 / USB-6361 specifications

<https://www.ni.com/docs/en-US/bundle/pcie-pxie-usb-6361-specs/page/specs.html>

Counter resources, timebases and accuracy; Table 4 PCIe PFI reference sources. Online revision 23-Jun-2025.

[11] NI X Series User Manual, 370784K-01, May 2019

<https://docs-be.ni.com/bundle/pcie-pxie-usb-63xx-features/raw/resource/enus/370784k.pdf>

Printed pp.7-28-7-31: triggered pulse generation; start/pause resource constraint. Chapters 7–9: counter and clock routing.

[12] NI Python API: counter-output pulse channel

https://nidaqmx-python.readthedocs.io/en/stable/task_collections.html#nidaqmx.task.collections.COChannelCollection.add_co_pulse_chan_time

Pulse-channel API. Installed Python signatures were inspected without creating a task or touching hardware.

[13] Thorlabs DC2200 Operation Manual, v1.6, 23-Sep-2020; corroborating manufacturer PDF mirror

<https://d17eythm3w95tp.cloudfront.net/media/3772/73a6cee4822eb31057abd5af2c3a1dc2.pdf>

Corroborates Pulse/PWM monitor direction and TTL level control. Older than [9].

Local audit, 15 September 2026

DAQ commit: 4dc71fbfe8b2dcbf27b1feaf90855c4022b98913.

daq/measurements/timestream.py:269–293, 368–410; daq/instruments/dc2200.py:147, 238–293, 385–394, 475–510; daq/instruments/_visa.py:575–650; daq/_base.py:33–86; daq/time_utils.py. Installed Presto 2.17.1 source reviewed as on p.10.

Notebook base: C:/Users/AQD/Documents/GitHub/analysis_archive/DAQ/

PRIMA_JKID_JPLQPD_20260831/JPLQPD/

B260416-NG-D2_MultitoneQC_LED_20260911.ipynb: source lines 130–162, 874–936, 1210–1238, 1446, 2785–2818.

B260416-NG-D2_KickNLED_20260902.ipynb: 211–219, 256–269, 338.

Saved evidence under the same base: data/multitone_qc_led/

20260915T204734Z_3eb90b91/manifest.json:213–236, 310–317.

Historical NI evidence: analysis_archive/DAQ/NIDAQ/StdFreqSweep.ipynb:68–69, 252–255.

Recommended next step

Validate a common Presto start, then test finite NI counter pulses into pre-armed DC2200 TTL. If the NI card is unavailable, use equivalent hardware timing or a Pulsed redesign; if physical alignment is unnecessary, capture actual pulse times and align analysis per record. I found no justified one-line fix for the current software-started internal LED train. No measurement code or instrument state was changed.